Agent Completeness via Circuit Simulation: A Natural Proof that Transformer Agents are Universal Computers

Greg Coppola, PhD
coppola.ai

March 2026

Abstract

We prove that a constant-depth transformer operating in a read-compute-write agent loop is Turing complete. Unlike prior constructions that encode instruction-set computers into transformer weights via artificial state-smuggling mechanisms, our proof proceeds through Boolean circuit simulation: we show that a 4-layer, 2-head transformer can simulate one depth-1 layer of any Boolean circuit in a single forward pass. The agent loop—read state, apply transformer, write state, repeat—provides unbounded iteration, yielding Turing completeness.

The construction uses attention for its natural purpose (content-addressable memory access to gather gate inputs from arbitrary positions) and feedforward layers for their natural purpose (computing local gate functions). The circuit state is represented as a sequence of typed propositions—each token encoding a bit's value, position, gate type, and wiring—connecting the construction to the *characteristica universalis* framework for formal reasoning. We argue that this constitutes evidence that attention over typed propositions is a natural computational primitive for universal computation, not merely a mechanism that can be forced to compute through artificial encodings.

1 Introduction

1.1 The Question

Can transformer-based agents compute any computable function? The answer is known to be yes. Pérez, Barceló, and Marinkovic (2021) established that transformers with sufficient depth or recursion can simulate Turing machines. Giannou et al. (2023) constructed explicit looped transformers of depth 9–13 that emulate one-instruction-set computers, calculators, linear algebra routines, and in-context learning algorithms. Wei, Chen, and Ma (2022) proved Turing completeness for bounded-precision transformers with recursive links.

The question is not *whether* transformers can compute, but *how naturally* they do so.

1.2 The Problem with Prior Constructions

Existing Turing completeness proofs for transformers share a common strategy: reverse-engineer the attention mechanism to perform operations it was not designed for. Giannou et al. encode program counters in positional encodings, carry intermediate state through scratchpad columns, implement binary arithmetic via carefully chosen key-query matrices, and construct 9-layer transformers that execute SUBLEQ instructions. The resulting transformer *works*—it provably simulates a one-instruction-set computer—but the attention mechanism is not doing anything recognizable as attention. It is being forced to behave like a CPU through artificial weight configurations.

This matters beyond aesthetics. If the goal is to understand *why* transformers are effective computational devices, a proof that treats attention as an arbitrary matrix multiplication misses the point. The question is whether there is a proof where the transformer computes naturally—where attention does what attention does.

1.3 Our Approach

We give a Turing completeness proof based on Boolean circuit simulation. The key insight is a clean separation of concerns:

1. **Bounded computation** is handled by the transformer. A single forward pass simulates one layer of a Boolean circuit: attention gathers gate inputs from arbitrary positions, and the feedforward network computes gate functions locally.

2. **Unbounded computation** is handled by the agent loop. The agent reads the current state, applies the transformer, writes the output as the new state, and repeats. This loop iterates circuit layers, providing the unbounded depth needed for Turing completeness.

This separation matches how agents actually operate: they think for a bounded time (one transformer pass), act on the environment (write the output), observe the result (read it back), and think again.

1.4 Connection to the Characteristica Universalis

The circuit state in our construction is represented as a sequence of typed propositions: each token encodes a gate's value, position, type, and wiring information. This representation connects directly to the *characteristica universalis* framework introduced in Coppola (2024, 2026a, 2026b), where reasoning is formalized as belief propagation over typed predicate structures.

The correspondence is:

Circuit	**Characteristica**	**Transformer**
Gate value	Proposition truth value	Token embedding (value slot)
Gate position	Proposition identifier	Positional encoding
Wiring ($j \mapsto i_1, i_2$)	Role binding	Attention pattern
Gate type (AND/OR/NOT)	Inference rule	FFN computation
Circuit layer	One inference step	One forward pass
Depth-D circuit	D-step derivation	D agent iterations

The proof thus supports a stronger claim than mere Turing completeness: the *natural* operations of attention—content-addressable lookup and local computation—over *typed propositions* are sufficient for universal computation.

1.5 Contributions

1. A **4-layer, 2-head transformer construction** that simulates one depth-1 layer of any fan-in-2 Boolean circuit (Theorem 5.1).

2. A **Turing completeness proof** via the agent loop that is simpler than prior constructions and uses attention for its natural purpose (Theorem 5.2).

3. A **formal connection** between circuit simulation over typed propositions and the characteristica universalis framework, providing evidence that attention over typed predicates is a natural primitive for universal computation.

1.6 Outline

Section 2 reviews transformer architecture and Boolean circuits. Section 3 defines the typed proposition encoding of circuit states. Section 4 presents the 4-layer construction. Section 5 states and proves the main theorems. Section 6 compares with prior work. Section 7 develops the connection to the characteristica universalis. Section 8 discusses limitations and open problems.

2 Preliminaries

2.1 Transformer Architecture

We follow the standard encoder-style transformer layer as defined in Vaswani et al. (2017) and used in Giannou et al. (2023). The input to each layer is $X \in \mathbb{R}^{d\times n}$, a sequence of n tokens each of dimension d.

A single-head attention layer computes:

$$\mathrm{Attn}(X) = X + VX\,\sigma_S\Big((KX)^\top(QX)\Big) \tag{1}$$

where $Q, K \in \mathbb{R}^{d_h\times d}$ are the query and key projection matrices, $V \in \mathbb{R}^{d\times d}$ is the value projection, and σ_S denotes column-wise softmax with temperature λ:

$$[\sigma_S(Z,\lambda)]_{i,j} = \frac{e^{\lambda Z_{i,j}}}{\sum_k e^{\lambda Z_{k,j}}}.$$

A transformer layer combines attention with a feedforward network:

$$f(X) = \mathrm{Attn}(X) + W_2\,\mathrm{ReLU}(W_1\,\mathrm{Attn}(X) + b_1\mathbf{1}_n^\top) + b_2\mathbf{1}_n^\top \tag{2}$$

where W_1, W_2 are weight matrices and b_1, b_2 are bias vectors.

A looped transformer applies the same multi-layer network $\mathrm{TF}(W;\cdot)$ iteratively:

$$X^{(t+1)} = \mathrm{TF}(W; X^{(t)}), \quad t = 0, 1, 2, \ldots \tag{3}$$

2.2 Boolean Circuits

Definition 2.1 (Boolean circuit)**.** *A* Boolean circuit *over n variables is a directed acyclic graph where:*

- *Input nodes are labeled $x_1, \ldots, x_n$.*
- *Internal nodes (gates) are labeled with a Boolean function: AND, OR, NOT, NAND, NOR, XOR, or COPY.*
- *Each gate has fan-in at most 2.*
- *A subset of nodes are designated as outputs $y_1, \ldots, y_m$.*

Definition 2.2 (Circuit depth and width)**.** *The* depth *of a circuit is the length of the longest path from any input to any output. The* width *at depth ℓ is the number of gates at depth ℓ. The* size *of a circuit is the total number of gates.*

Definition 2.3 (Depth-1 layer)**.** *A circuit of depth D can be decomposed into D* depth-1 layers *$L_1, \ldots, L_D$, where layer L_ℓ consists of all gates at depth ℓ. Each gate in L_ℓ depends only on outputs of gates in $L_{\ell-1}$ (or original inputs for $\ell = 1$). A depth-1 layer is a function $L : \{0,1\}^n \to \{0,1\}^n$ where each output bit is a single gate applied to at most two input bits.*

The following is standard (see, e.g., Arora and Barak, 2009):

Theorem 2.4 (Circuit universality)**.** *For every Turing machine M that halts on all inputs within $T(n)$ steps, there exists a uniform circuit family $\{C_n\}$ of depth $O(T(n))$ and size $O(T(n)\log T(n))$ that computes the same function as M.*

3 Typed Proposition Encoding

3.1 State Representation

We represent the state of a Boolean circuit as a sequence of n tokens, where each token encodes one wire (bit) of the circuit. Each token is a column vector $\mathbf{x}_j \in \mathbb{R}^d$ with the following structure:

$$\mathbf{x}_j = \begin{bmatrix} v_j \\ \mathbf{pos}_j \\ \mathbf{gate}_j \\ \mathbf{wire}_j^{(1)} \\ \mathbf{wire}_j^{(2)} \\ \mathbf{scratch}_j \end{bmatrix} \tag{4}$$

where:

- $v_j \in \{0, 1\}$ is the current bit value.
- $\mathbf{pos}_j \in \{-1, +1\}^b$ is the binary positional encoding of index j, with $b = \lceil \log_2 n \rceil$.
- $\mathbf{gate}_j \in \{0, 1\}^G$ is a one-hot vector indicating the gate type (G is the number of gate types: AND, OR, NOT, NAND, NOR, XOR, COPY).
- $\mathbf{wire}_j^{(1)} \in \{-1, +1\}^b$ is the positional encoding of gate j's first input.
- $\mathbf{wire}_j^{(2)} \in \{-1, +1\}^b$ is the positional encoding of gate j's second input (equal to $\mathbf{wire}_j^{(1)}$ for unary gates).
- $\mathbf{scratch}_j \in \mathbb{R}^s$ is a scratch register for intermediate values, initialized to zero.

The total embedding dimension is $d = 1 + b + G + 2b + s = 1 + 3b + G + s$. With $s = 2$ (to store two gathered input values), we have $d = 3 + 3\lceil \log_2 n \rceil + G$.

3.2 Typed Proposition Interpretation

Each token $\mathbf{x}_j$ corresponds to a typed proposition in the characteristica universalis:

$$\texttt{gate(pos:}\ j,\ \texttt{val:}\ v_j,\ \texttt{type:}\ g_j,\ \texttt{in1:}\ i_1(j),\ \texttt{in2:}\ i_2(j)\texttt{)}$$

The circuit state $X = [\mathbf{x}_1, \ldots, \mathbf{x}_n]$ is a knowledge base of n typed propositions. One step of circuit evaluation—reading inputs and computing gate outputs—corresponds to one step of inference over this knowledge base.

3.3 Positional Encoding Properties

Following Giannou et al. (2023), we use binary positional encodings $\mathbf{pos}_j \in \{-1, +1\}^b$ where $b = \lceil \log_2 n \rceil$. These satisfy:

$$\mathbf{pos}_i^\top \mathbf{pos}_j = \begin{cases} b & \text{if } i = j \\ \leq b - 2 & \text{if } i \neq j \end{cases} \tag{5}$$

since any two distinct binary vectors differ in at least one coordinate. With softmax temperature $\lambda = \Omega(b)$, the attention weight concentrates on the matching position:

$$\sigma_S(\mathbf{pos}_i^\top \mathbf{pos}_j)_i \geq 1 - (n-1)e^{-2\lambda} \geq 1 - \epsilon \tag{6}$$

for $\lambda \geq \frac{1}{2} \log \frac{n}{\epsilon}$. In the following, we write $\approx$ to denote equality up to error ϵ, which can be made arbitrarily small by increasing λ.

4 The 4-Layer Construction

We now construct a 4-layer, 2-head transformer that simulates one depth-1 layer of any fan-in-2 Boolean circuit. Each layer performs a single conceptual operation.

4.1 Layer 1: Gather First Input

Goal. For each column j, retrieve the value $v_{i_1(j)}$ from position $i_1(j)$ (the first input wire) and store it in the scratch register.

Attention head. Define:

$$Q_1 = \begin{bmatrix} \mathbf{0}_{b\times 1} & \mathbf{0}_{b\times b} & \mathbf{0}_{b\times G} & I_b & \mathbf{0}_{b\times b} & \mathbf{0}_{b\times s} \end{bmatrix} \tag{7}$$

$$K_1 = \begin{bmatrix} \mathbf{0}_{b\times 1} & I_b & \mathbf{0}_{b\times G} & \mathbf{0}_{b\times b} & \mathbf{0}_{b\times b} & \mathbf{0}_{b\times s} \end{bmatrix} \tag{8}$$

Then $Q_1\mathbf{x}_j = \mathbf{wire}_j^{(1)}$ (the positional encoding of the first input) and $K_1\mathbf{x}_j = \mathbf{pos}_j$ (the position of column j). The attention score between query j and key ℓ is:

$$(Q_1\mathbf{x}_j)^\top(K_1\mathbf{x}_\ell) = (\mathbf{wire}_j^{(1)})^\top \mathbf{pos}_\ell$$

By the concentration property (6), the softmax output is approximately $\mathbf{e}_{i_1(j)}$—a one-hot vector selecting column $i_1(j)$.

Value matrix. Define V_1 to extract the value v and place it in the first scratch slot. Concretely, $V_1 \in \mathbb{R}^{d\times d}$ has a single nonzero entry: row $(d-s+1)$, column 1, equal to 1. This maps the value v_ℓ of the attended column into the first scratch position of column j.

Result. After the residual connection, column j is:

$$\mathbf{x}_j \leftarrow \mathbf{x}_j + V_1\mathbf{x}_{i_1(j)} \approx \begin{bmatrix} v_j \\ \mathbf{pos}_j \\ \mathbf{gate}_j \\ \mathbf{wire}_j^{(1)} \\ \mathbf{wire}_j^{(2)} \\ v_{i_1(j)} \\ 0 \end{bmatrix}$$

Feedforward. The FFN of layer 1 is the identity (all weights zero). No additional computation is needed.

Lemma 4.1. *Layer 1, as defined by* (7)–(8) *and the value matrix above, copies* $v_{i_1(j)}$ *into the first scratch slot of every column* j*, with error at most* ϵ *for temperature* $\lambda \geq \frac{1}{2}\log\frac{n}{\epsilon}$.

Proof. The query $Q_1\mathbf{x}_j = \mathbf{wire}_j^{(1)}$ matches the key $K_1\mathbf{x}_\ell = \mathbf{pos}_\ell$ maximally when $\ell = i_1(j)$, by the positional encoding properties (5). The value projection V_1 extracts the scalar v_ℓ and places it in scratch slot 1. The residual connection adds this to $\mathbf{x}_j$, which has zero in scratch slot 1 by initialization. Error is bounded by (6). □

4.2 Layer 2: Gather Second Input

Goal. For each column j, retrieve $v_{i_2(j)}$ and store it in the second scratch slot.

The construction is identical to Layer 1, except:

$$Q_2 = \begin{bmatrix} \mathbf{0}_{b\times 1} & \mathbf{0}_{b\times b} & \mathbf{0}_{b\times G} & \mathbf{0}_{b\times b} & I_b & \mathbf{0}_{b\times s} \end{bmatrix} \tag{9}$$

so $Q_2\mathbf{x}_j = \mathbf{wire}_j^{(2)}$, and V_2 places the retrieved value in scratch slot 2.

Lemma 4.2. *Layer 2 copies $v_{i_2(j)}$ into the second scratch slot of every column j, with error at most ϵ.*

Proof. Identical to Lemma 4.1, with $\mathbf{wire}_j^{(2)}$ replacing $\mathbf{wire}_j^{(1)}$. □

State after Layer 2. Each column j now contains:

$$\mathbf{x}_j \approx \begin{bmatrix} v_j \\ \mathbf{pos}_j \\ \mathbf{gate}_j \\ \mathbf{wire}_j^{(1)} \\ \mathbf{wire}_j^{(2)} \\ v_{i_1(j)} \\ v_{i_2(j)} \end{bmatrix} \tag{10}$$

Both inputs to gate j are now locally available.

4.3 Layer 3: Compute Gate Function

Goal. Compute $g_j(v_{i_1(j)}, v_{i_2(j)})$ and write the result into v_j.

Attention. The attention head of layer 3 is set to the identity (value matrix $V_3 = 0$), so the layer reduces to a pure feedforward computation.

Feedforward network. Each column j contains the gate type $\mathbf{gate}_j$ (one-hot) and both input values v_{i_1}, v_{i_2} in the scratch register. We need to compute:

$$y_j = \sum_{t=1}^{G} [\mathbf{gate}_j]_t \cdot f_t(v_{i_1(j)}, v_{i_2(j)}) \tag{11}$$

where f_t is the Boolean function for gate type t.

Each f_t can be implemented exactly by a ReLU network with $O(1)$ neurons:

Lemma 4.3. *For inputs $a, b \in \{0, 1\}$, the following ReLU expressions compute the standard Boolean gates:*

$$\mathrm{AND}(a, b) = \mathrm{ReLU}(a + b - 1.5)/0.5 \tag{12}$$
$$\mathrm{OR}(a, b) = \mathrm{ReLU}(a + b - 0.5) - \mathrm{ReLU}(a + b - 1.5) \tag{13}$$
$$\mathrm{NOT}(a) = 1 - a \tag{14}$$
$$\mathrm{NAND}(a, b) = 1 - \mathrm{AND}(a, b) \tag{15}$$
$$\mathrm{XOR}(a, b) = \mathrm{OR}(a, b) - \mathrm{AND}(a, b) \tag{16}$$
$$\mathrm{COPY}(a) = a \tag{17}$$

Proof. Direct verification on $\{0, 1\}^2$. For AND: when $a = b = 1$, $\mathrm{ReLU}(0.5)/0.5 = 1$; for all other inputs $a+b \leq 1$, so $\mathrm{ReLU}(\leq -0.5) = 0$. For OR: when $a+b \geq 1$, $\mathrm{ReLU}(a+b-0.5) > 0$ and the second term corrects the case $a = b = 1$ to output 1. The others follow by composition. □

To implement the gated selection (11), we exploit the fact that $\mathbf{gate}_j$ is one-hot. The FFN computes all G gate functions in parallel and uses the gate type as a selector:

$$y_j = \sum_{t=1}^{G} \mathrm{ReLU}\big([\mathbf{gate}_j]_t + f_t(v_{i_1}, v_{i_2}) - 1.5\big) \cdot 2 \tag{18}$$

This is nonzero only for the active gate type t^* where $[\mathbf{gate}_j]_{t^*} = 1$ and $f_{t^*} \in \{0, 1\}$, giving $y_j = f_{t^*}(v_{i_1}, v_{i_2})$.

The FFN then writes y_j to the value slot and zeros out the scratch register. The total FFN width for this layer is $O(G)$—a constant independent of circuit size.

Lemma 4.4. *The feedforward network of layer 3, with $O(G)$ hidden neurons, computes $g_j(v_{i_1(j)}, v_{i_2(j)})$ for every column j in parallel and writes the result to the value slot.*

Proof. Each gate function f_t requires $O(1)$ ReLU neurons (Lemma 4.3). The gated selection (18) requires one additional ReLU per gate type. With G gate types, the total hidden layer width is $O(G)$. The output layer writes to the value slot and zeros the scratch register. Since all columns share the same FFN weights and the gate type selects the appropriate function per column, the computation is correct for all n columns in parallel. □

4.4 Layer 4: Update Wiring for Next Iteration

Goal. Prepare the state for the next agent iteration by loading the wiring information for the next circuit layer.

There are two approaches:

Approach A: Preloaded program. The full circuit description—wiring for all D layers—is stored in additional "program" tokens appended to the sequence. Layer 4's attention head reads the wiring for the next layer (indexed by an iteration counter stored in a distinguished token) and overwrites $\mathbf{wire}_j^{(1)}$ and $\mathbf{wire}_j^{(2)}$.

Approach B: Fixed circuit. If the same depth-1 layer is applied repeatedly (as in the simulation of a Turing machine's transition function, which has fixed structure), the wiring is already correct and layer 4 is a no-op.

For the general case (Approach A), we require one attention head to read from the program tokens. The iteration counter is maintained in a distinguished token and incremented by the FFN using the binary addition construction of Giannou et al. (2023, Lemma 16): a single ReLU hidden layer with $O(b)$ neurons can add 1 to a b-bit binary counter.

Lemma 4.5. *Layer 4, with one attention head and $O(b)$ FFN neurons, updates the wiring information for the next circuit layer and increments the iteration counter.*

Proof. The attention head uses the iteration counter as a query to select the next layer's wiring from the program tokens, following the same gather mechanism as Layers 1–2. The FFN implements binary increment of the counter using $O(b)$ neurons. □

5 Main Results

Theorem 5.1 (Circuit layer simulation)**.** *Let $L : \{0,1\}^n \to \{0,1\}^n$ be a depth-1 Boolean circuit layer with fan-in 2 and G gate types. There exists a transformer T with:*

- *4 layers*
- *2 attention heads (one in layers 1–2 each, zero effective heads in layer 3, one in layer 4)*
- *Embedding dimension $d = O(\log n + G)$*
- *FFN width $O(G + \log n)$*
- *Temperature $\lambda = O(\log(n/\epsilon))$*

such that $T(X) = X'$ where X encodes the input and circuit description and X' encodes the output, with per-bit error at most ϵ.

Proof. Compose Layers 1–4 as constructed in Section 4. Layer 1 gathers the first input (Lemma 4.1). Layer 2 gathers the second input (Lemma 4.2). Layer 3 computes the gate function (Lemma 4.4). Layer 4 updates wiring for the next iteration (Lemma 4.5). Error accumulates additively across the two attention operations in layers 1–2, each with error at most $\epsilon/2$, for total error ϵ. The gate computation in layer 3 is exact on binary inputs. □

Theorem 5.2 (Agent completeness)**.** *For any computable function $f : \{0,1\}^* \to \{0,1\}^*$, there exists a 4-layer transformer T and an encoding E such that the agent loop*

$$S_{t+1} = T(S_t), \quad S_0 = E(x, C_f)$$

computes $f(x)$ after D iterations, where D is the depth of the circuit C_f for f and $E(x, C_f)$ encodes both the input x and the circuit description.

Proof. By Theorem 2.4, f is computed by a circuit family $\{C_n\}$ of depth $D(n)$. Decompose C_n into $D(n)$ depth-1 layers. By Theorem 5.1, one transformer pass simulates one depth-1 layer. After $D(n)$ iterations of the agent loop, all layers have been simulated, producing $f(x)$. □

Corollary 5.3 (Turing completeness)**.** *A 4-layer transformer agent operating in a read-compute-write loop is Turing complete.*

Proof. A Turing machine M with state set Q, tape alphabet Γ, and transition function δ can be simulated step-by-step. Each TM step (read symbol, look up transition, write symbol, move head, update state) is a fixed-depth Boolean circuit over the tape encoding. The circuit depth per TM step is $O(1)$ (independent of tape length, since δ has fixed size). By Theorem 5.2, the transformer agent simulates one TM step per constant number of agent iterations. The agent loop runs indefinitely, simulating arbitrarily many TM steps. □

6 Comparison with Prior Work

6.1 Giannou et al. (2023)

Giannou et al. construct looped transformers that simulate a one-instruction-set computer (OISC). Their SUBLEQ transformer requires 9 layers and 2 heads; their extended FLEQ framework requires up to 13 layers. The construction encodes program counters in positional encodings, carries intermediate state in a scratchpad, implements binary subtraction via feedforward layers, and reverse-engineers attention to perform memory read/write operations.

Our construction differs in three respects. First, we use fewer layers (4 vs. 9–13) because circuit simulation is conceptually simpler than OISC emulation. Second, each layer of our construction performs a single natural operation—gather or compute—whereas their layers combine multiple operations within a single SUBLEQ cycle. Third, our attention heads perform content-addressable lookup (query a position, retrieve a value), which is the natural function of attention. Their attention heads implement memory transfer operations that, while valid, are an unnatural use of the mechanism.

Construction	**Layers**	**Heads**	**Width**	**Attention role**
Giannou et al. SUBLEQ	9	2	$O(\log n + N)$	Memory transfer
Giannou et al. FLEQ	$9 + \max l_i$	$\sum h_i$	$O(Md + \log n)$	Memory transfer
This work	4	2	$O(\log n + G)$	Content-addressable lookup

6.2 Pérez et al. (2021)

Pérez, Barceló, and Marinkovic proved that transformers with hard attention or infinite precision are Turing complete. Their construction requires unbounded precision or depth proportional to the computation length. Our construction uses bounded precision (the softmax temperature λ depends only on $\log n$, not on computation length) and constant depth, with the agent loop providing unbounded computation.

6.3 Wei et al. (2022)

Wei, Chen, and Ma proved Turing completeness for bounded-precision transformers with recursive links. Their construction is closest to ours in using a loop for unbounded computation, but their internal mechanism encodes a Turing machine directly rather than proceeding through circuit simulation. Our circuit-based approach yields a simpler proof and a more interpretable construction.

7 Connection to the Characteristica Universalis

7.1 The Formal Correspondence

The construction in Section 4 is not merely a proof technique—it instantiates a structural correspondence between Boolean computation, the characteristica universalis of Coppola (2024, 2026a, 2026b), and the transformer architecture.

Definition 7.1 (Circuit state as knowledge base)**.** *A circuit state $X = [\mathbf{x}_1, \ldots, \mathbf{x}_n]$ corresponds to a knowledge base $\mathcal{K}$ of n typed propositions:*

$$\mathcal{K} = \left\{ \texttt{gate}(\texttt{pos:}\, j,\ \texttt{val:}\, v_j,\ \texttt{type:}\, g_j,\ \texttt{in1:}\, i_1(j),\ \texttt{in2:}\, i_2(j)) \;\middle|\; j = 1, \ldots, n \right\}$$

Definition 7.2 (Circuit evaluation as inference)**.** *One depth-1 layer of circuit evaluation corresponds to one step of inference over $\mathcal{K}$:*

1. ***Universal instantiation**: the wiring $(i_1(j), i_2(j))$ binds the gate's input roles to specific propositions, analogous to grounding a universally quantified rule.*
2. ***Modus ponens**: the gate function g_j applies a deterministic rule to derive the output value, analogous to a Horn clause firing.*
3. ***Knowledge base update**: the derived values replace the old values, producing a new knowledge base $\mathcal{K}'$.*

Proposition 7.3. *The transformer construction of Theorem 5.1 implements one step of forward-chaining inference over a knowledge base of typed propositions, where:*

- *Attention implements universal instantiation (role binding via content-addressable lookup).*
- *The feedforward network implements modus ponens (rule application via gate computation).*
- *The agent loop implements iterated inference (repeated forward chaining).*

7.2 Evidence for Structural Convergence

The characteristica universalis thesis (Coppola, 2026b) claims that typed predicates with role labels are the uniquely natural formal language for reasoning, supported by three arguments: historical convergence, parsimony, and completeness.

The construction in this paper provides a fourth argument: **computational convergence**. The transformer architecture—designed for language modeling via gradient descent, with no explicit logical structure—implements computation over typed propositions naturally. The attention mechanism performs predicate binding (matching queries to keys, as predicates match role-labeled arguments). The feedforward network performs knowledge lookup and rule application. The layer structure corresponds to inference depth.

This convergence from radically different starting points—axiomatic logic (Frege through Prawitz) and stochastic gradient descent (word2vec through GPT)—suggests that typed propositions are not merely a convenient encoding but the natural representation for computation over structured knowledge.

8 Discussion

8.1 What the Proof Shows

Our proof establishes three results:

1. **Technically**: a 4-layer, 2-head transformer in an agent loop is Turing complete. This improves on the 9–13 layers of Giannou et al. (2023) and provides a simpler proof than Pérez et al. (2021).

2. **Architecturally**: the construction uses attention for content-addressable lookup and feed-forward layers for local computation—the natural functions of each component. Prior constructions use attention for state smuggling, which obscures the relationship between the architecture and the computation.

3. **Conceptually**: the circuit state can be interpreted as a knowledge base of typed propositions, and circuit evaluation as forward-chaining inference. This connects transformer computation to the characteristica universalis and supports the thesis that attention over typed propositions is a natural primitive for universal computation.

8.2 Limitations

Sequence length. Our construction requires sequence length $n + O(S)$ where S is the circuit size, to store both the current state and the circuit description. For the Turing machine simulation in Corollary 5.3, the sequence length must accommodate the tape, which may grow unboundedly. In practice, an agent can request additional memory (longer context) as needed, but our formal model assumes a fixed sequence length per invocation.

Softmax approximation. The attention mechanism uses softmax, which only approximates the hard selection required for exact circuit simulation. The error can be made arbitrarily small by increasing the temperature λ, but cannot be made exactly zero. Following Giannou et al. (2023), an error correction layer using ReLU rounding can be added if exact computation is required, at the cost of one additional layer.

Naturalness. We claim that our construction is "more natural" than prior work. This claim has a precise component (attention performs content-addressable lookup rather than state transfer) and an interpretive component (the characteristica framing). The precise component is verifiable by inspecting the weight matrices. The interpretive component is a thesis, not a theorem.

8.3 Open Problems

Belief propagation simulation. A stronger result would show that a transformer agent can simulate one step of belief propagation over a Logical Bayesian Network factor graph. This would directly connect transformer computation to the calculus ratiocinator, closing the loop between the three papers of Coppola (2024, 2026a, 2026b). We conjecture that this is provable with a similar construction, where attention aggregates messages from factor neighbors and the FFN computes belief updates.

Trained transformers. Our construction uses hardcoded weights. An important open question is whether transformers trained on language data learn weight configurations that approximate the circuit simulation structure—whether trained attention heads perform content-addressable lookup over proposition-like representations and trained FFN layers compute rule-like functions. Preliminary evidence from mechanistic interpretability research is suggestive but not conclusive.

Optimal depth. We have not attempted to minimize the number of layers. It is plausible that a 2-layer construction suffices (one gather layer with two heads, one compute layer), but we have not verified this.

References

[1] Arora, S. and Barak, B. (2009). *Computational Complexity: A Modern Approach*. Cambridge University Press.

[2] Coppola, G. (2024). The quantified boolean Bayesian network: Theory and experiments with a logical graphical model. *arXiv preprint arXiv:2402.06557*.

[3] Coppola, G. (2026a). Statistical parsing for logical information retrieval.

[4] Coppola, G. (2026b). The universal language: A characteristica universalis for AI.

[5] Giannou, A., Rajput, S., Sohn, J., Lee, K., Lee, J. D., and Papailiopoulos, D. (2023). Looped transformers as programmable computers. *arXiv preprint arXiv:2301.13196*.

[6] Minsky, M. L. (1961). Recursive unsolvability of Post's problem of "tag" and other topics in theory of Turing machines. *Annals of Mathematics*, 74(3):437–455.

[7] Pérez, J., Barceló, P., and Marinkovic, J. (2021). Attention is Turing-complete. *Journal of Machine Learning Research*, 22(75):1–35.

[8] Sipser, M. (2012). *Introduction to the Theory of Computation*. Cengage Learning, 3rd edition.

[9] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.

[10] Wei, C., Chen, Y., and Ma, T. (2022). Statistically meaningful approximation: a case study on approximating Turing machines with transformers. *Advances in Neural Information Processing Systems*.